AGENT EVALUATION
SCIENCE

Evaluating Agent Swarms: Token Budgets, Active Workers, and Baseline Choice

Maximilian Nicholson[1,2] Ayda Yazdani[1]
[1] University of Bath [2] Amazon
mn866@bath.ac.uk ay582@bath.ac.uk

Abstract

Evaluating an agent swarm requires distinguishing the effects of increasing the total token budget, dividing a fixed ceiling, and changing coordination. We examine these comparisons in 8,400 controlled trials on fresh arithmetic and graph-colouring tasks. At 4,096 completion tokens per worker, increasing independent width from one to four improves accuracy by 16.0 percentage points on both families while the total ceiling quadruples. Splitting a fixed 4,096-token completion ceiling instead lowers graph accuracy from 56.0% to 18.5%; the arithmetic change is inconclusive. The primary comparisons do not establish an accuracy advantage over a sequential control matched on scheduled calls and nominal token ceiling. Ring communication provides no consistent gain. Eleven complementary scaling studies add 20,376 held-out trials across seven task families, including a fresh-task sixteen-worker confirmation that does not establish superiority over fixed-four execution. Candidate and request audits show that finding a valid answer after an initial failure depends on the token allowance. They also reveal incomplete worker activation and 97 controlled-study attempts with unknown usage. The findings show how resource controls, output selection, and execution records qualify claims about agent scaling. They support evaluating specific allocation policies; equal realized compute advantages and general scaling laws remain unestablished from this work.

Keywords: Agent evaluation; Multi-agent systems; Test-time compute; Token budgets; Resource allocation; Coordination; Reliability; Reproducibility

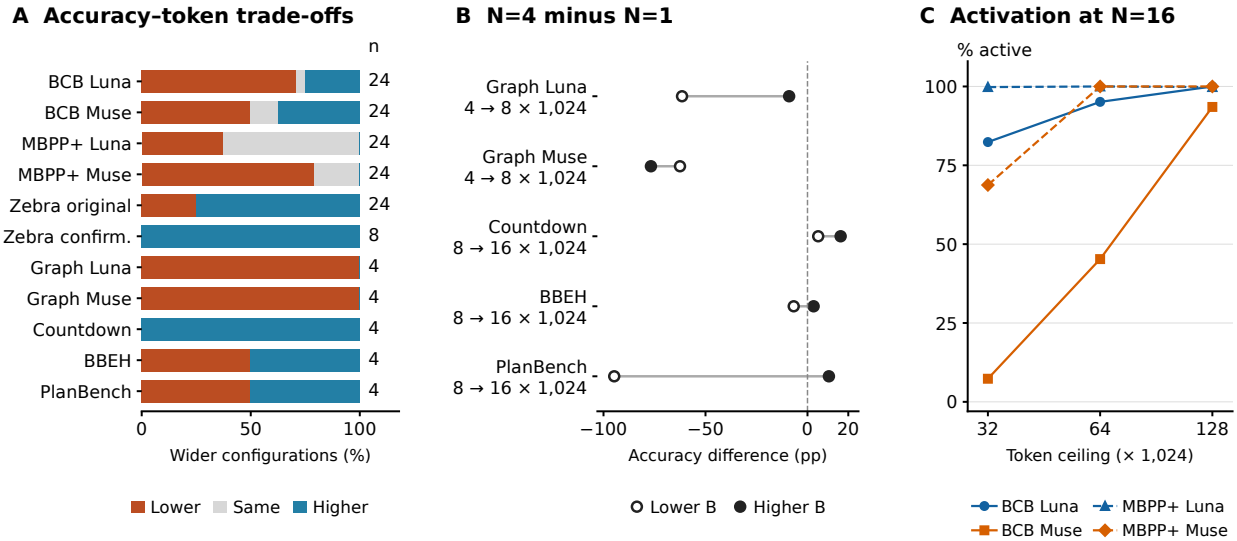


Figure 1 | (A) Wider-cell accuracy versus matched-ceiling $N = 1$ (148 configurations sharing tasks). (B) Four-worker effects at two ceilings, pooled over revision conditions. (C) Activation at sixteen requested workers.

1 Study and Findings

This ongoing study distinguishes sampling, aggregation, and debate (Brown et al., 2024; Wang et al., 2023; Du et al., 2023), with allocation controls for task and architecture dependence (Snell et al., 2024; Kim et al., 2026). Controlled widths are $N = 1, 2, 4$ (Figure 2); programming/ZebraLogic grids use $N = 1, 2, 4, 8, 16$, diagnostics $N = 1, 4$. Task sources: BigCodeBench, MBPP/EvalPlus, ZebraLogic, BBEH, PlanBench (Zhuo et al., 2025; Austin et al., 2021; Liu et al., 2023; Lin et al., 2025; Kazemi et al., 2025; Valmeekam et al., 2023). Graph/Countdown tasks use deterministic checks; graphs adapt Reasoning Gym (Stojanovski et al., 2025). Paired bootstrap resamples task clusters, stratified where applicable (Efron, 1979; Field and Welsh, 2007).

All 148 wider cells (Figure 1) use more recorded tokens: 75 score lower, 24 equal, 49 higher. BigCodeBench Muse activation at $N = 16$ is 7.3%, 45.3%, and 93.5% at ceilings 32,768, 65,536, and 131,072. ZebraLogic $4/8/16$ minus fixed four is $+0.92$ pp (95% CI $[-0.58, 2.50]$), establishing neither superiority nor equivalence. Controlled fixed-total arithmetic: -24.0 pp, inconclusive.

At $B = 32{,}768$, independent BigCodeBench Muse passes 40/64 ($N = 1$), 36/64 ($N = 8$), and 14/64 ($N = 16$). Programming supplies 96 wider cells, 81 flat or worse; counts are not benchmark-balanced.

PlanBench's independent four-worker effect changes from -95.0 pp ($[-98.0, -91.5]$) to $+10.5$ pp ($[6.0, 15.5]$) as its ceiling doubles (higher-ceiling debit ratio: 4.06). BBEH ring effects change from -6.5 to $+4.5$ pp. Countdown's independent gains of 9.5–17.0 pp use 2.77–3.85 times the recorded tokens; its pooled ring effect is -5.0 pp ($[-8.5, -1.75]$). Our ring protocol is narrower than shared-workspace communication (Park et al., 2026).

Resource audits complement failure taxonomies (Cemri et al., 2025). Unknown usage in 97 controlled attempts limits accounting; equal ceilings do not establish equal compute. Ongoing work tests matched resources and policy transfer; general scaling laws remain unestablished. Evidence and protocols: https://github.com/PluvioXO/agent-swarms-reproducibility.

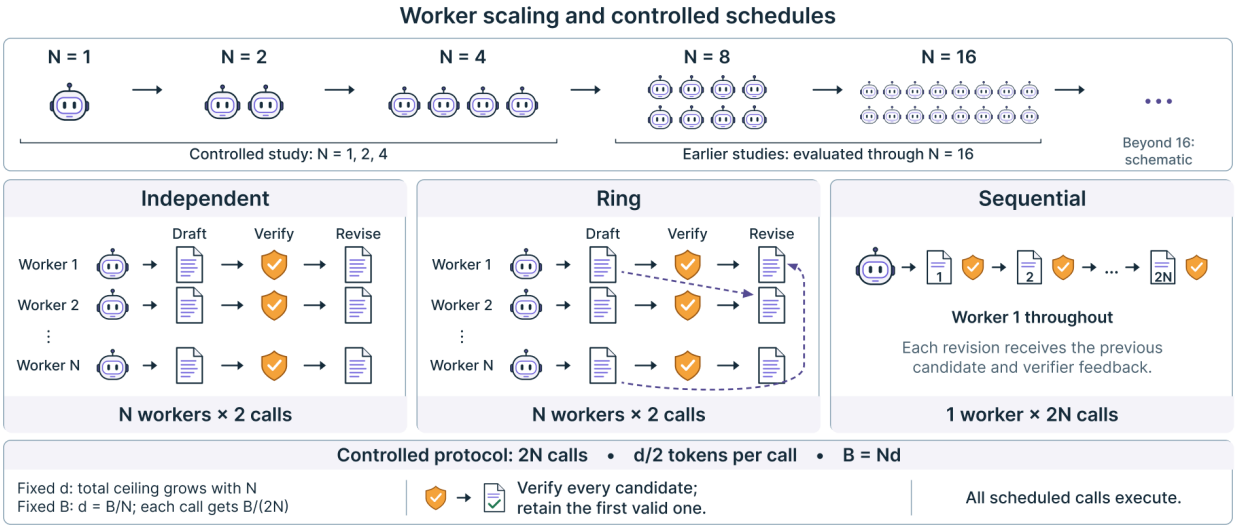


Figure 2 | Architectures and allocation: width N, per-worker completion allowance d, total ceiling $B = Nd$. All $2N$ calls execute with $d/2$ tokens per call. Controlled widths: 1, 2, 4; earlier studies: through 16. Larger widths are schematic.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, et al. Program synthesis with large language models, 2021. https://arxiv.org/abs/2108.07732. Cited at: 1.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, et al. Large language monkeys: Scaling inference compute with repeated sampling, 2024. https://arxiv.org/abs/2407.21787. Cited at: 1.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, et al. Why do multi-agent llm systems fail?, 2025. https://arxiv.org/abs/2503.13657. Cited at: 1.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. https://arxiv.org/abs/2305.14325. Cited at: 1.

Bradley Efron. 1979. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26. https://doi.org/10.1214/aos/1176344552. Cited at: 1.

C. A. Field and A. H. Welsh. 2007. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390. https://doi.org/10.1111/j.1467-9868.2007.00593.x. Cited at: 1.

Mehran Kazemi, Bahare Fatemi, Hritik Bansal, John Palowitch, Chrysovalantis Anastasiou, Sanket Vaibhav Mehta, et al. Big-bench extra hard, 2025. https://arxiv.org/abs/2502.19187. Cited at: 1.

Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, et al. Towards a science of scaling agent systems, 2026. https://arxiv.org/abs/2512.08296v3. Cited at: 1.

Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, et al. Zebralogic: On the scaling limits of llms for logical reasoning, 2025. https://arxiv.org/abs/2502.01100. Cited at: 1.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023. https://arxiv.org/abs/2305.01210. Cited at: 1.

Jongho Park, Vasilis Kontonis, Shivam Garg, Akshay Krishnamurthy, and Dimitris Papailiopoulos. Scaling discovery through test-time communication, 2026. https://arxiv.org/abs/2609.21032. Cited at: 1.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. https://arxiv.org/abs/2408.03314. Cited at: 1.

Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, et al. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards, 2025. https://arxiv.org/abs/2505.24760. Cited at: 1.

Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change, 2023. https://arxiv.org/abs/2206.10498. Cited at: 1.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, et al. Self-consistency improves chain of thought reasoning in language models, 2023. https://arxiv.org/abs/2203.11171. Cited at: 1.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions, 2025. https://arxiv.org/abs/2406.15877. Cited at: 1.